

# AETHER

## Enterprise Operations Portal & System Manual

---

*Production-Grade CRM, Dynamic Inventory Management,  
and Automated Commercial Sales Challan Workflow Engine*

### ARCHITECTURAL SUMMARY

Backend: Node.js / Express.js / TypeScript / Prisma ORM / SQLite  
Frontend: React / Vite / TypeScript / Custom Vanilla CSS (No Tailwind)  
Security: JSON Web Token (JWT) Authorization / Bcrypt Password Hashing  
Integrity: Transaction-Safe Stock Locking & Real-Time Price/Client Snapshots

### PREPARED FOR:

Senior Technical Recruitment & Operations Evaluators

### REPOSITORY SUBMISSION:

GitHub: <https://github.com/fenilpatel417-code/aether>  
Live Demo: Deployed on Render Cloud Hosting Platform

## 1. Project Mission & Executive Summary

---

The Aether Operations Portal is an enterprise-grade ERP and CRM hub designed to automate wholesale supply-chain distribution pipelines. The application replaces insecure paper trails, scattered communication logs, and manual spreadsheets with an atomic, transactional ledger system. Aether connects four critical corporate divisions (System Administration, Sales, Warehouse Operations, and Accounts Management) under a single secure console.

By enforcing strict role-based access control (RBAC), automatic serial code generation, real-time inventory safety check triggers, and immutable invoice snapshotting, Aether guarantees corporate ledger integrity, preventing data decay and transaction mismatch over time.

## 2. Functional Specifications by Division

---

### 2.1 Director & System Administration

Administrators hold complete horizontal visibility across all system components. They audit high-level KPI trends (sales volume growth, safety stock breach percentages), register/modify wholesale products, override stock levels, log customer follow-up notes, and issue or cancel transactions.

### 2.2 Sales Executive Division

Sales operatives drive the customer onboarding pipeline. They register new client profiles, transition CRM accounts from 'Leads' to 'Active', and log meeting notes on the timeline ledger. They can compose Sales Challans in 'Draft' state (which holds 0 stock in reserve) or trigger confirmation requests.

### 2.3 Warehouse Management Division

Warehouse users monitor stock safety levels. The interface alerts them to products falling below minimum safety bounds. They log incoming shipments (increasing stock) and specify reasons for manual quantity adjustments. All edits write entries in the stock audit trail.

## 2.4 Finance & Accounts Controller

Accounts officers handle the transactional validation gate. They audit confirmed sales challans, export formatted PDF invoices, and review client tax records (GSTIN). They can cancel confirmed challans, which triggers an automated rollback transaction that restores product quantities in the warehouse.

## 3. System Architecture & Data flow

---

Aether is designed as a modular monorepo. It features a stateless REST API backend communicating with a responsive, animation-driven Single Page Application (SPA) frontend.

### 3.1 Full-Stack Technology Stack

- **Frontend Client:** React 18 with Vite build framework. Managed entirely using Vanilla CSS variables and modern layout layouts (Flexbox/Grid) to ensure lightweight, framework-agnostic execution with zero style overhead.
- **Backend Server:** Node.js with Express.js written in TypeScript. Code compilation is strictly managed via a localized tsc compiler structure configured for CommonJS runtime portability.
- **Database Layer:** SQLite engine integrated using Prisma ORM. Prisma abstracts database connections, enabling the system to transition from SQLite to PostgreSQL or MySQL simply by editing schema properties.
- **Authorization:** Stateless JSON Web Tokens (JWT) signed using HS256. User credentials are encrypted at rest using the bcryptjs blowfish block cipher.

### 3.2 Data Snapshotting Pattern for Financial Integrity

A critical problem in standard inventory databases is value corruption over time. If a product's price updates or a customer changes their billing address, historical sales challans will display incorrect values if they rely on active relational joins.

Aether solves this by serializing and saving a complete JSON snapshot of the 'Customer' and 'Product' data inside the 'SalesChallan' and 'SalesChallanItem' tables at the exact millisecond of confirmation. This guarantees that historical records remain immutable and accurate for audits, even if catalog items are modified or deleted.

## 4. Relational Database Schema

Below are the primary relational entities structured in the database schema. These columns are enforced using Prisma constraints:

### 4.1 Customer Entity

Stores demographic and tax identifiers. A customer can have multiple follow-up notes and challans.

| Field        | Type          | Nullability | Description                        |
|--------------|---------------|-------------|------------------------------------|
| id           | String (UUID) | Primary Key | Unique identifier                  |
| name         | String        | Required    | Full customer name                 |
| businessName | String        | Required    | Wholesale business enterprise name |
| gstNumber    | String        | Optional    | Tax ID (15-character GSTIN)        |
| customerType | String        | Required    | Retail   Wholesale   Distributor   |
| address      | String        | Required    | Shipping & billing address details |
| status       | String        | Required    | Lead   Active   Inactive           |

### 4.2 Product & Stock Log Entity

Stores products and safety parameters. Any change in 'currentStock' automatically appends an audit entry inside the StockLog table.

| Field         | Type          | Nullability | Description                      |
|---------------|---------------|-------------|----------------------------------|
| id            | String (UUID) | Primary Key | Unique product ID                |
| name          | String        | Required    | Product catalog label            |
| sku           | String        | Unique      | Stock Keeping Unit (indexed)     |
| unitPrice     | Float         | Required    | Base sale price                  |
| currentStock  | Int           | Required    | Current quantity in warehouse    |
| minStockAlert | Int           | Required    | Threshold for low-stock triggers |
| location      | String        | Required    | Shelf/aisle address in warehouse |

## 5. API Contract Specification

The REST interface is documented below. Token authorization must be supplied for secure routes.

### 5.1 Authentication (POST /api/auth/login)

Generates session token for authorization.

```
Request Body:
{
  "username": "admin",
  "password": "admin123"
}
Response (200 OK):
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "c8b67b1f-7b0f-48d6-bd0e-a61ef563e41b",
    "username": "admin",
    "name": "Operations Director",
    "role": "Admin"
  }
}
```

### 5.2 Create Sales Challan (POST /api/challans)

Compiles order. Confirmed status checks stock; Draft bypasses reservations.

```
Request Headers:
Authorization: Bearer <token>
Request Body:
{
  "customerId": "a18b76fc-8b6e-41df-a720-3df138e6e589",
  "status": "Confirmed",
  "items": [
    { "productId": "p8f7c9e0-8b6a-4c28-98de-56fa2b87c012", "quantity": 12, "unitPrice": 450.00 }
  ]
}
Response (210 Created):
{
  "id": "ch76f8e9-82a1-432d-98bc-a16f290d238c",
  "challanNumber": "CHLN-20260812-0001",
  "status": "Confirmed",
  "totalQuantity": 12,
  "totalAmount": 5400.00
}
```

## 6. Deployment & Operations Manual

Aether is engineered for zero-dependency local running and direct integration with cloud platforms like Render.

### 6.1 Quick-Start Local Installation

Follow this command sequence to boot the backend and frontend nodes locally in development mode:

```
# 1. Initialize and Seed SQLite database
cd backend
npm install
npx prisma db push
npx prisma db seed
npm run dev      # Backend starts on http://localhost:5000

# 2. Start Frontend web client (in secondary terminal tab)
cd ../frontend
npm install
npm run dev      # Frontend starts on http://localhost:5173
```

### 6.2 Render Cloud Platform Configurations

To publish Aether live on the Web, configure two separate entities on the Render Dashboard:

- **Aether API Web Service:** Connect your GitHub repository. Set Root Directory to 'backend'. Set Build Command to: 'npm install && npx prisma db push && npm run build && npx prisma db seed'. Set Start Command to: 'npm start'. Add env variables: 'JWT\_SECRET' and 'PORT'.
- **Aether SPA Client Static Site:** Connect the same repository. Set Root Directory to 'frontend'. Set Build Command to: 'npm install && npm run build'. Set Publish Directory to 'dist'. Add env variable: 'VITE\_API\_URL' pointing to your backend URL (no trailing slash).

### 6.3 System Verification & Validation Tests

To prove transaction integrity after deployment, run the following manual test script:

- **Verify Stock Lock:** Select a product with 10 units in stock. Log in as 'sales' and attempt to confirm a challan of 12 units. Verify the system blocks the confirm action and triggers a warning toast.
- **Verify Reversion:** Create a confirmed challan of 5 units. Observe that stock drops to 5. Log in as 'accounts', open the challan, and click 'Cancel'. Verify that stock immediately increments back to 10 and writes a stock log entry.
- **Verify Snapshotting:** Generate a confirmed challan for a customer. Open the CRM page and change that customer's business address. Open the challan detailed modal or download the PDF invoice; verify that the billing address renders the original shipping address captured during invoice creation.